ANALIZADOR SINTÁCTICO PASCAL – C

DOCUMENTACIÓN DEL ASIGNACIÓN ENTRE LOS INTEGRANTES

ALGORITMO DE COLECCIÓN CANÓNICA



Equipo 6

Integrantes:

Sánchez Aguilar Luis Ángel
Álvarez Olivera Irving Damián
Ramírez Manzano Jaret
García Barela Ángel Andrés
López Osorio Haniel
Martinez Cesar Zaid
Soriano Martínez Miguel Ángel

Materia: Compiladores

Profesor: David Martínez Torres

Fecha de entrega: Viernes 16 de octubre del 2026

UNIVERSIDAD TECNOLÓGICA DE LA MIXTECA

Índice

1. Introducción
2. Descripción general del sistema
3. Objetivo del programa
4. Arquitectura del sistema
    - 4.1. Entrada y validación de la gramática
    - 4.2. Representación de los elementos LR(0)
    - 4.3. Algoritmo de cerradura
    - 4.4. Algoritmo Ir_a
    - 4.5. Generación de la colección canónica
    - 4.6. Interfaz gráfica
    - 4.7. Manejo de errores y generación de resultados
5. Tecnologías utilizadas
6. Flujo de funcionamiento
7. Conclusiones
8. Tabla de asignaciones
9. Asignaciones de la semana

1. Introducción

El presente documento describe el desarrollo de un programa cuyo propósito es generar la colección canónica de elementos LR(0), procedimiento que forma parte de la construcción de analizadores sintácticos en el área de compiladores.

El programa permite procesar una gramática de entrada y obtener los diferentes estados que conforman su colección canónica, así como las transiciones que existen entre ellos. Para lograrlo, se implementan los algoritmos de cerradura e Ir_a, los cuales permiten calcular los conjuntos de elementos necesarios para construir los estados.

Asimismo, el sistema integra mecanismos para validar la gramática, presentar los resultados mediante una interfaz gráfica y comprobar el funcionamiento de los algoritmos a través de diferentes casos de prueba.

2. Descripción general del sistema

El sistema funciona como una herramienta de apoyo para la construcción de la colección canónica de elementos LR(0). Su funcionamiento parte de una gramática definida mediante producciones y continúa con el procesamiento de sus símbolos y la generación de los estados correspondientes.

El programa se organiza en diferentes módulos, cada uno encargado de una operación específica. Esta separación permite desarrollar y verificar los algoritmos de manera independiente, para posteriormente integrarlos en un solo sistema.

De manera general, el procesamiento sigue el siguiente esquema:

Gramática de entrada → Validación → Cerradura e Ir_a → Generación de estados y transiciones → Visualización de resultados

El resultado es una colección de estados sin duplicados, acompañada de las transiciones calculadas entre ellos. Esta información permite observar cómo se estructura la gramática desde la perspectiva del análisis sintáctico.

3. Objetivo del programa

El objetivo principal del programa es automatizar la construcción de la colección canónica de elementos LR(0) a partir de una gramática de entrada, aplicando los procedimientos necesarios para generar sus estados y transiciones.

Entre sus principales funciones se encuentran:

- Leer y validar los componentes de una gramática.
- Representar las producciones mediante elementos LR(0).
- Calcular la cerradura de un conjunto de elementos.
- Aplicar la función Ir_a para obtener estados sucesores.
- Generar la colección canónica sin repetir estados equivalentes.
- Registrar las transiciones entre los estados generados.
- Mostrar los resultados mediante una interfaz gráfica.
- Comprobar el funcionamiento del programa con diferentes gramáticas de prueba.

4. Arquitectura del sistema

La arquitectura del programa se organiza en módulos que trabajan de manera coordinada. Cada componente recibe información de una etapa anterior, realiza una operación específica y proporciona los resultados necesarios para continuar con el proceso.

Esta organización permite separar el procesamiento de la gramática, los cálculos de los algoritmos y la presentación de resultados, facilitando la revisión y el mantenimiento del sistema.

4.1. Entrada y validación de la gramática

Esta etapa se encarga de cargar la gramática que será procesada por el programa. A partir del archivo de entrada, se identifican sus componentes principales: terminales, no terminales, símbolo inicial y producciones.

También se verifica que la información proporcionada tenga una estructura compatible con el formato esperado. Cuando corresponde, se comprueba la presencia de la producción aumentada, necesaria para iniciar la construcción de la colección canónica.

Entrada: archivo con la definición de una gramática.

Salida: estructura de datos que contiene los símbolos y las producciones validadas, o información sobre los errores detectados.

4.2. Representación de los elementos LR(0)

Para construir los estados de la colección canónica, las producciones de la gramática se representan mediante elementos LR(0). Cada elemento contiene una producción y un punto que indica hasta qué posición se ha reconocido su lado derecho durante el análisis.

Por ejemplo, una producción como (A \rightarrow aB) puede representarse mediante los siguientes elementos:

- (A \rightarrow \cdot aB)
- (A \rightarrow a \cdot B)
- (A \rightarrow aB \cdot)

El desplazamiento del punto permite determinar qué símbolo puede procesarse en una transición y cuándo se ha completado una producción.

Esta representación constituye la base para calcular las cerraduras, las transiciones y los estados de la colección canónica.

4.3. Algoritmo de cerradura

El algoritmo de cerradura, denominado Cerradura(I), recibe un conjunto de elementos LR(0) y obtiene todos los elementos que deben pertenecer a dicho conjunto según las producciones de la gramática.

Cuando el punto de un elemento se encuentra inmediatamente antes de un no terminal, se incorporan las producciones de ese no terminal con el punto al inicio. Este procedimiento se repite hasta que no sea necesario agregar nuevos elementos.

La cerradura permite completar los estados con la información necesaria para representar las posibles continuaciones de las producciones.

Entrada: conjunto inicial de elementos LR(0).

Salida: conjunto ampliado con todos los elementos correspondientes a su cerradura.

4.4. Algoritmo Ir_a

La función Ir_a(I, X) permite calcular el estado que se obtiene al avanzar sobre un símbolo determinado (X), a partir de un conjunto de elementos (I).

Para ello, se identifican los elementos cuyo punto se encuentra inmediatamente antes del símbolo indicado y se desplaza el punto una posición hacia la derecha. Posteriormente, se aplica la cerradura al conjunto resultante para completar el nuevo estado.

Si ningún elemento permite avanzar sobre el símbolo seleccionado, la función devuelve un conjunto vacío.

Este algoritmo es fundamental para establecer las transiciones entre los estados de la colección canónica.

Entrada: conjunto de elementos LR(0) y símbolo de transición.

Salida: conjunto de elementos correspondiente al estado sucesor, o un conjunto vacío cuando no existe transición.

4.5. Generación de la colección canónica

La generación de la colección canónica utiliza los resultados de los algoritmos de cerradura e Ir_a para construir todos los estados alcanzables a partir de la gramática.

El proceso comienza con la cerradura de la producción inicial aumentada. Después, se recorren los símbolos que aparecen después del punto en los elementos de cada estado y se calcula la transición correspondiente mediante Ir_a.

Cada estado obtenido se compara con los estados existentes. Si representa un conjunto nuevo de elementos, se incorpora a la colección para procesarlo posteriormente. Si ya existe, se reutiliza el estado correspondiente, evitando duplicaciones.

El procedimiento continúa hasta que no quedan estados nuevos por procesar. Durante el recorrido se registran las transiciones entre estados y se identifica el

estado de aceptación cuando contiene el elemento completo de la producción inicial aumentada.

Entrada: gramática validada y algoritmos de cerradura e Ir_a.

Salida: colección canónica de estados y registro de sus transiciones.

4.6. Interfaz gráfica

La interfaz gráfica permite interactuar con el programa sin necesidad de ejecutar manualmente cada algoritmo. A través de ella, el usuario puede cargar la gramática, iniciar el procesamiento y consultar los resultados generados.

La información se presenta de manera organizada para facilitar la identificación de los estados y de los elementos que contiene cada uno. También se contempla la presentación de mensajes que permitan reconocer problemas relacionados con la entrada de datos o la ejecución del programa.

La interfaz funciona como medio de comunicación entre el usuario y los módulos encargados de realizar los cálculos.

4.7. Manejo de errores y generación de resultados

El sistema contempla la revisión de la información de entrada y la presentación de resultados al finalizar el procesamiento.

Cuando la gramática presenta datos faltantes o inconsistencias con el formato esperado, el programa debe informar el problema para facilitar su corrección. De igual manera, durante las pruebas se revisa que las funciones devuelvan resultados coherentes ante casos sin transiciones o conjuntos vacíos.

Una vez completado el algoritmo, se muestran los estados de la colección canónica y las transiciones generadas, permitiendo comprobar la estructura obtenida.

Resultado: colección canónica calculada, transiciones registradas y mensajes de validación o error cuando corresponda.

5. Tecnologías utilizadas

Para el desarrollo del sistema se emplearon herramientas de programación y estructuras de datos que permiten implementar los algoritmos y presentar sus resultados.

Tecnología o componente	Función
Lenguaje de programación utilizado (Python)	Implementar los algoritmos y controlar el flujo de procesamiento.
Estructuras de datos	Representar las producciones, los elementos LR(0), los estados y las transiciones.
Interfaz gráfica	Facilitar la carga de gramáticas y la visualización de los resultados.
Archivos de gramáticas	Proporcionar los datos de entrada para ejecutar y comprobar el programa.
Algoritmos de cerradura e Ir_a	Calcular los conjuntos de elementos y las transiciones necesarios para generar la colección canónica.

6. Flujo de funcionamiento

El funcionamiento general del programa puede describirse mediante las siguientes etapas:

1. El usuario proporciona un archivo que contiene la gramática.
2. El programa lee y valida los símbolos y las producciones.
3. Las producciones se representan mediante elementos LR(0).
4. Se calcula la cerradura de la producción inicial aumentada para obtener el primer estado.
5. Se aplica Ir_a sobre los símbolos correspondientes para generar estados sucesores.
6. Se comparan los estados obtenidos con los existentes para evitar duplicados.
7. Se registran las transiciones y se continúa procesando los estados nuevos hasta completar la colección.
8. Se presentan los estados y las transiciones mediante la interfaz gráfica.
9. Se verifican los resultados con las gramáticas de prueba y los valores de referencia.

De esta manera, el programa transforma la gramática de entrada en una colección estructurada de estados y transiciones que puede utilizarse como base para la construcción de un analizador sintáctico LR(0).

7. Conclusiones

El desarrollo del programa permitió implementar los procedimientos fundamentales para la generación de la colección canónica de elementos LR(0). Mediante la integración de la lectura de gramáticas, los algoritmos de cerradura e Ir_a y el recorrido de los estados, se estableció un proceso automatizado para obtener la colección y sus transiciones.

La separación del sistema en módulos facilitó la organización de las operaciones y permitió realizar pruebas sobre los distintos componentes antes de verificar su funcionamiento conjunto.

Finalmente, las pruebas con diferentes gramáticas y la comparación de los resultados con cálculos manuales constituyeron una parte importante del proceso de validación. El proyecto representa una aplicación práctica de los conceptos estudiados en la materia de Compiladores y contribuye a comprender cómo se construyen los estados que intervienen en el análisis sintáctico.

8. Tabla de asignaciones

RESPONSABLE	ACTIVIDAD	DESCRIPCIÓN	ENTREGABLE
Ángel Andrés García Barela	**Colección canónica**	Implementar el algoritmo que construye el estado inicial, recorre los estados y símbolos, utiliza `Ir_a` y agrega únicamente estados nuevos. Registrar las transiciones y la aceptación conforme al algoritmo del profesor. Integrar los módulos al proyecto existente.	Algoritmo de colección canónica; estados y transiciones sin duplicados.
Ángel Andrés García Barela	**Lectura y validación de la gramática**	Cargar el archivo y obtener terminales, no terminales, símbolo inicial y producciones. Validar sus componentes y la producción aumentada. Se reutiliza el lector existente.	Módulo de carga; estructura de la gramática; mensajes de error claros; archivos de ejemplo compatibles.
Irving Damián Álvarez Olivera	**Algoritmo Cerradura**	Implementar `cerradura(I)`, agregando los elementos necesarios hasta que no aparezcan nuevos. Manejar recursión y producciones vacías.	Modelo de elemento; algoritmo Cerradura; ejemplos verificados, incluidos casos con `ε`.
Haniel López Osorio	**Algoritmo Ir_a y comparación de estados**	Implementar `ir_a(I, X)`: desplazar el punto sobre el símbolo indicado y aplicar Cerradura. Comparar estados como conjuntos, para reconocer estados iguales aunque sus elementos tengan distinto orden.	Algoritmo Ir_a; comparación de estados; pruebas de transiciones y resultados vacíos.
Jaret Ramírez Manzano	**Interfaz gráfica**	Incorporar funcionalidad a la GUI existente. Permitir cargar la gramática, ejecutar el cálculo y visualizar la gramática y cada estado con sus elementos. Mostrar errores de entrada de manera comprensible.	Ventana o pestaña funcional; controles conectados al algoritmo; presentación legible de los resultados.
Miguel Ángel Soriano Martínez	**Pruebas y verificación de resultados**	Preparar y comprobar las pruebas g1, g2 y g3 de la rúbrica. Contrastar elementos y transiciones con resultados calculados manualmente. Probar también las demás gramáticas de la asignación y reportar fallos.	Archivos de prueba; resultados esperados; registro de pruebas; evidencias y lista de errores encontrados.
Cesar Zaid Martinez	**UML e informe**	Actualizar el UML conforme a la implementación real. Reunir las explicaciones y evidencias de cada integrante y preparar el informe con índice.	Diagrama de clases actualizado; informe con índice, explicación de módulos y evidencias.
Luis Ángel Sánchez Aguilar	**Integración**	…	…

9.Itinerario de actividades de la semana

Día	Objetivo principal	Tareas asignadas del día	Descripción específica de las actividades	Responsables
Lunes 12	Organización y análisis del proyecto	Revisar los requisitos, distribuir las actividades y preparar las gramáticas de prueba.	Se revisó el funcionamiento de la colección canónica y se identificaron los componentes necesarios para implementarla. Se organizaron las tareas de lectura de gramáticas, cerradura, Ir_a, generación de estados, interfaz gráfica y pruebas. También se revisaron las gramáticas que se utilizarían para comprobar los resultados.	Todo el equipo
Martes 13	Desarrollo de los algoritmos fundamentales	Implementar la lectura y validación de la gramática, la cerradura y la función Ir_a.	Se trabajó en la lectura de los archivos de entrada para obtener los terminales, no terminales, símbolo inicial y producciones. Se desarrolló la función Cerradura(I) para incorporar los elementos correspondientes a los no terminales situados después del punto. Asimismo, se implementó Ir_a(I, X) para desplazar el punto sobre un símbolo y obtener el conjunto resultante, incluyendo la comparación de estados.	Angel Andres Garcia Barela Irving Damian Alvarez Olivera Haniel Lopez Osorio
Miércoles 14	Construcción de la colección e integración	Generar los estados y las transiciones, evitar duplicados e integrar los algoritmos con la interfaz gráfica.	Se implementó el recorrido de los estados de la colección canónica, comenzando por la cerradura de la producción aumentada. Se calcularon las transiciones mediante Ir_a, se incorporaron los estados nuevos y se verificó que los estados equivalentes no se agregaran nuevamente. Paralelamente, se avanzó en la conexión de los algoritmos con la interfaz para cargar gramáticas y visualizar los resultados.	Angel Andres Garcia Barela Jaret Ramirez Manzano Irving Damian Alvarez Olivera

Jueves 15	Pruebas y corrección de errores	Ejecutar las pruebas g1, g2 y g3, comparar los resultados con cálculos manuales y corregir inconsistencias.	Se ejecutaron las gramáticas de prueba y se revisaron los estados generados, sus elementos y las transiciones obtenidas. Los resultados se contrastaron con cálculos manuales para identificar diferencias. También se revisaron casos especiales, como conjuntos vacíos, producciones vacías y estados repetidos, realizando ajustes en los módulos que presentaban inconsistencias.	Miguel Angel Soriano Martinez Cesar Zaid Martinez
Viernes 16	Verificación y preparación de la entrega	Revisar el funcionamiento general, organizar los archivos de prueba y reunir las evidencias para la documentación.	Se verificó el funcionamiento integrado del programa, desde la lectura de la gramática hasta la presentación de la colección canónica. Se revisó que los estados y las transiciones fueran consistentes con los resultados esperados y que la interfaz mostrara la información de forma comprensible. Finalmente, se organizaron los archivos de prueba, los resultados obtenidos y la documentación del proyecto para preparar la entrega.	Todo el equipo